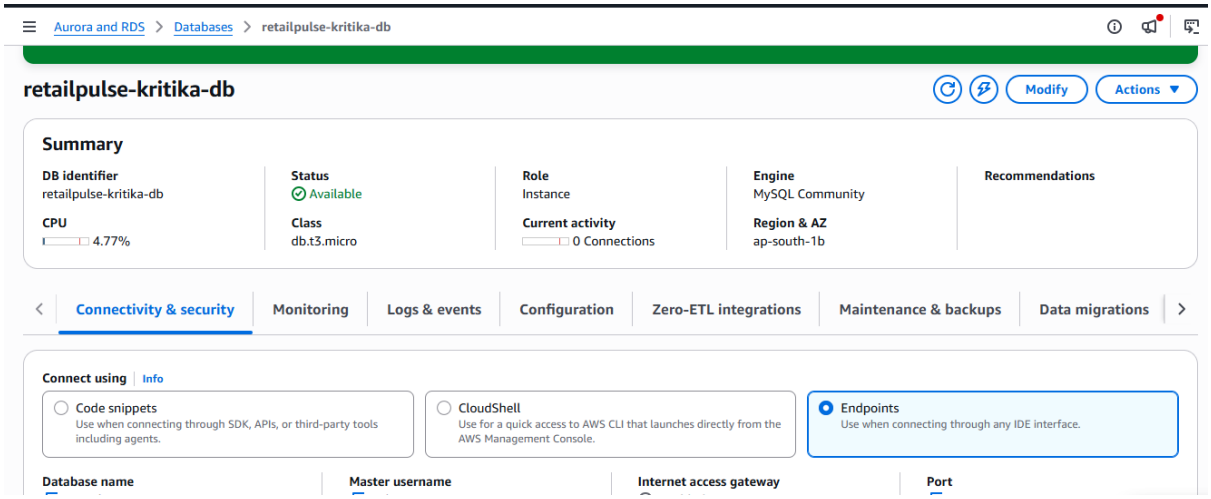
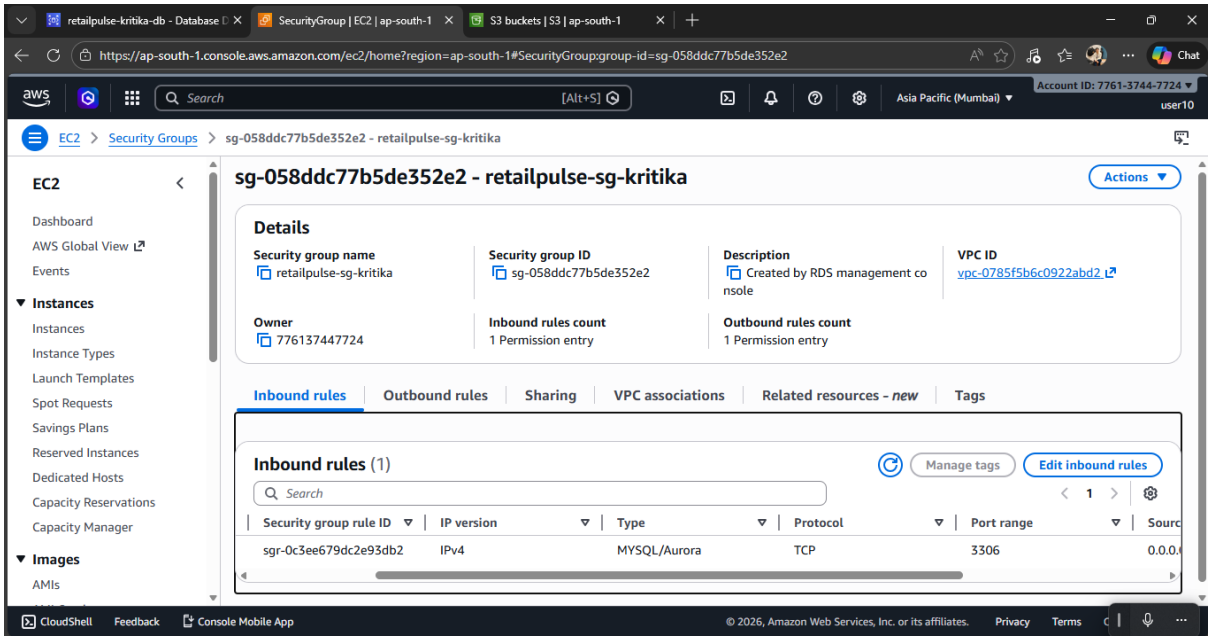




Now create an instance :

Loading the data to the ec2 instance:

```
PS C:\Users\kritika.nimje\Downloads> history

  Id CommandLine
  -- -----------
   1 cd Downloads
   2 scp -i /path/to/Kritika_ec2.pem /path/to/customers.csv ec2-user...
   3 scp -i "Kritika_ec2.pem" customers.csv ec2-user@13.203.160.9:/h...
   4 cp -i "kritika-key.pem" products.csv ec2-user@13.203.160.9:/hom...
   5 scp -i "Kritika_ec2.pem" products.csv ec2-user@13.203.160.9:/ho...
   6 scp -i "Kritika_ec2.pem" orders.csv ec2-user@13.203.160.9:/home...
   7 scp -i "Kritika_ec2.pem" distributors.csv ec2-user@13.203.160.9...
   8 scp -i "Kritika_ec2.pem" inventory_snapshots.csv ec2-user@13.20...
   9 ls
```

MySQL Command Execution Log – retailpulse_kritika

Environment: EC2 to RDS (MariaDB/MySQL)

Database Initialization

```
SHOW DATABASES;
CREATE DATABASE retailpulse_kritika;
USE retailpulse_kritika;
SELECT DATABASE();
```

Customers Table

```
CREATE TABLE customers (
  customer_id VARCHAR(30),
  registration_date DATE,
  city VARCHAR(80),
  state VARCHAR(50),
  pincode VARCHAR(10),
  customer_segment VARCHAR(20),
  email_verified BOOLEAN,
  phone_verified BOOLEAN,
  loyalty_tier VARCHAR(15),
  acquisition_channel VARCHAR(30),
  date_of_birth DATE,
  gender VARCHAR(10)
```

```
);
```

Products Table

```
CREATE TABLE products (
  product_id VARCHAR(30),
  product_name VARCHAR(200),
  category VARCHAR(50),
  sub_category VARCHAR(80),
  brand VARCHAR(80),
  base_mrp_inr DECIMAL(10,2),
  cost_price_inr DECIMAL(10,2),
  weight_kg DECIMAL(6,3),
  is_active BOOLEAN,
  launch_date DATE,
  reorder_threshold INT,
  return_window_days INT
);
```

Distributors Table

```
CREATE TABLE distributors (
  distributor_id VARCHAR(20),
  distributor_name VARCHAR(150),
  region VARCHAR(30),
  state VARCHAR(50),
  city VARCHAR(80),
  tier VARCHAR(15),
  contact_email VARCHAR(100),
  contact_phone VARCHAR(20),
  is_active BOOLEAN,
  onboarding_date DATE
);
```

Orders Table

```
CREATE TABLE orders (
  order_id VARCHAR(40),
```

```
  customer_id VARCHAR(30),
  order_date DATETIME,
  order_status VARCHAR(20),
  product_id VARCHAR(30),
  quantity INT,
  unit_price_inr DECIMAL(10,2),
  mrp_inr DECIMAL(10,2),
  discount_code VARCHAR(20),
  payment_method VARCHAR(20),
  delivery_pincode VARCHAR(10),
  distributor_id VARCHAR(20),
  platform VARCHAR(15),
  created_at DATETIME
);
```

Inventory Snapshots Table

```
CREATE TABLE inventory_snapshots (
  snapshot_id VARCHAR(40),
  snapshot_date DATE,
  distributor_id VARCHAR(20),
  product_id VARCHAR(30),
  opening_stock INT,
  units_received INT,
  units_dispatched INT,
  units_returned INT,
  units_damaged INT,
  closing_stock INT,
  reorder_point INT
);
```

CSV Load Commands

```
LOAD DATA LOCAL INFILE '/home/ec2-user/customers.csv'
INTO TABLE customers
FIELDS TERMINATED BY ','
LINES TERMINATED BY '
'
IGNORE 1 ROWS;
```

```
LOAD DATA LOCAL INFILE '/home/ec2-user/products.csv'
INTO TABLE products
FIELDS TERMINATED BY ','
LINES TERMINATED BY '
'
IGNORE 1 ROWS;

LOAD DATA LOCAL INFILE '/home/ec2-user/distributors.csv'
INTO TABLE distributors
FIELDS TERMINATED BY ','
LINES TERMINATED BY '
'
IGNORE 1 ROWS;

LOAD DATA LOCAL INFILE '/home/ec2-user/orders.csv'
INTO TABLE orders
FIELDS TERMINATED BY ','
LINES TERMINATED BY '
'
IGNORE 1 ROWS;

LOAD DATA LOCAL INFILE '/home/ec2-user/inventory_snapshots.csv'
INTO TABLE inventory_snapshots
FIELDS TERMINATED BY ','
LINES TERMINATED BY '
'
IGNORE 1 ROWS;
```

Verification Queries

```
SELECT COUNT(*) FROM customers;
SELECT COUNT(*) FROM products;
SELECT COUNT(*) FROM orders;
SELECT COUNT(*) FROM distributors;
SELECT COUNT(*) FROM inventory_snapshots;

SELECT * FROM orders WHERE order_id IS NULL;
SELECT * FROM products WHERE cost_price_inr <= 0;
```

SELECT * FROM inventory_snapshots WHERE closing_stock < 0;

STEP 3 – Data Validation Queries & Results (RetailPulse)

Database: retailpulse_kritika

Environment: EC2 → RDS (MySQL/MariaDB)

This document contains ALL validation queries executed during STEP 3, exactly as run in MySQL, along with their purpose.

1. Customers – Data Quality Validation

```
SELECT * FROM customers WHERE customer_id IS NULL;
```

```
SELECT customer_id, COUNT(*) FROM customers GROUP BY
customer_id HAVING COUNT(*) > 1;
```

```
SELECT * FROM customers WHERE state IS NULL;
```

2. Products – Pricing & Inventory Rules

```
SELECT * FROM products WHERE cost_price_inr <= 0;
```

```
SELECT * FROM products WHERE reorder_threshold < 0;
```

```
SELECT product_id, COUNT(*) FROM products GROUP BY product_id
HAVING COUNT(*) > 1;
```

3. Orders – Transaction Integrity (CRITICAL)

```
SELECT * FROM orders WHERE order_id IS NULL;
```

```
SELECT * FROM orders WHERE order_date > NOW();
```

```
SELECT * FROM orders o LEFT JOIN customers c ON o.customer_id
= c.customer_id WHERE c.customer_id IS NULL;
```

```
SELECT * FROM orders WHERE order_status = 'RETURNED' AND
order_id NOT IN (SELECT order_id FROM orders WHERE
order_status = 'DELIVERED');
```

```
SELECT * FROM orders WHERE unit_price_inr > mrp_inr;
```

```
SELECT o.* FROM orders o JOIN products p ON o.product_id =
p.product_id WHERE o.unit_price_inr < p.cost_price_inr;
```

```
SELECT * FROM orders WHERE delivery_pincode NOT REGEXP '^[1-
9][0-9]{5}$';
```

```
SELECT order_id, COUNT(*) FROM orders GROUP BY order_id HAVING
COUNT(*) > 1;
```

4. Inventory Snapshots – Stock & Alert Engine

```
SELECT * FROM inventory_snapshots WHERE closing_stock < 0;

SELECT * FROM inventory_snapshots WHERE closing_stock !=
(opening_stock + units_received - units_dispatched +
units_returned - units_damaged);

SELECT * FROM inventory_snapshots WHERE closing_stock = 0;

SELECT * FROM inventory_snapshots WHERE closing_stock <
reorder_point;
```

5. Conclusion

```
-- All above queries returned expected faulty records

-- Data quality rules, pricing rules, lifecycle rules, and
inventory alert logic verified

-- System ready for STEP 4: Transformations & Clean/Invalid
Data Routing
```

STEP 4 – Data Cleaning & Transformation Queries

Database: retailpulse_kritika

Environment: EC2 → RDS (MySQL / MariaDB)

Purpose: Separate clean vs invalid data and generate inventory alerts without overwriting raw tables.

4.1 Customers Cleaning

```
CREATE TABLE customers_clean AS SELECT * FROM customers WHERE
customer_id IS NOT NULL AND state IS NOT NULL;

CREATE TABLE customers_invalid AS SELECT * FROM customers
WHERE customer_id IS NULL OR state IS NULL;

SELECT COUNT(*) FROM customers_clean;

SELECT COUNT(*) FROM customers_invalid;
```

4.2 Products Cleaning

```
CREATE TABLE products_clean AS SELECT * FROM products WHERE
cost_price_inr > 0 AND reorder_threshold >= 0;
```

```
CREATE TABLE products_invalid AS SELECT * FROM products WHERE
cost_price_inr <= 0 OR reorder_threshold < 0;

SELECT COUNT(*) FROM products_clean;

SELECT COUNT(*) FROM products_invalid;
```

4.3 Orders Cleaning (Critical)

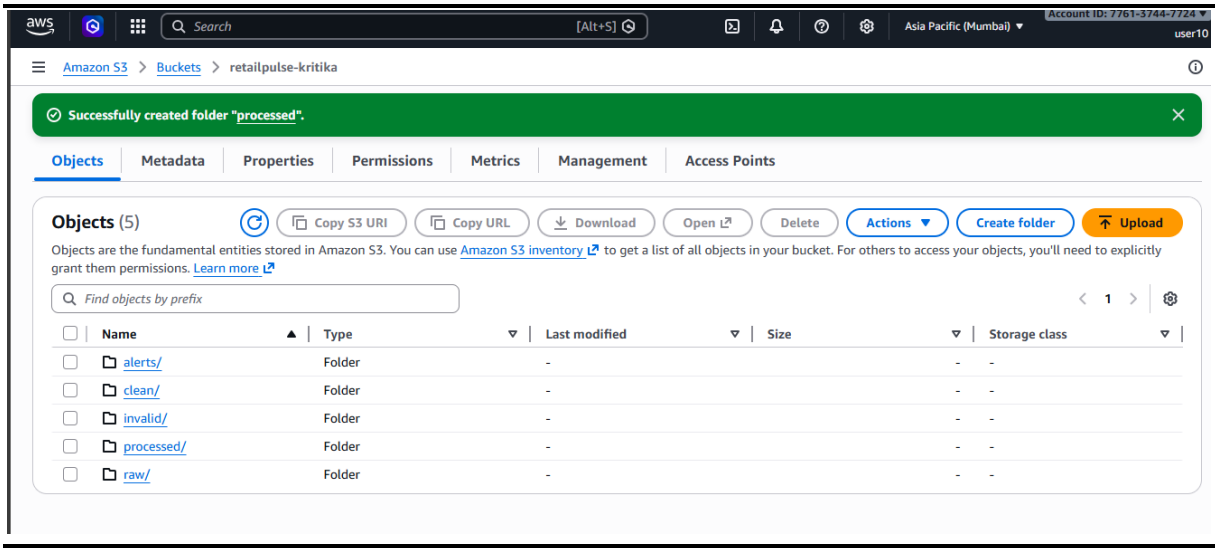
```
CREATE TABLE orders_clean AS SELECT o.* FROM orders o JOIN
customers_clean c ON o.customer_id = c.customer_id JOIN
products_clean p ON o.product_id = p.product_id WHERE
o.order_id IS NOT NULL AND o.order_date <= NOW() AND
o.unit_price_inr <= o.mrp_inr AND o.delivery_pincode REGEXP
'^[1-9][0-9]{5}$';

CREATE TABLE orders_invalid AS SELECT * FROM orders WHERE
order_id IS NULL OR order_date > NOW() OR unit_price_inr >
mrp_inr OR delivery_pincode NOT REGEXP '^[1-9][0-9]{5}$';

SELECT COUNT(*) FROM orders_clean;

SELECT COUNT(*) FROM orders_invalid;
```

4.4 Inventory Cleaning

```
CREATE TABLE inventory_clean AS SELECT * FROM
inventory_snapshots WHERE closing_stock >= 0 AND closing_stock
= (opening_stock + units_received - units_dispatched +
units_returned - units_damaged);

CREATE TABLE inventory_invalid AS SELECT * FROM
inventory_snapshots WHERE closing_stock < 0 OR closing_stock
!= (opening_stock + units_received - units_dispatched +
units_returned - units_damaged);

SELECT COUNT(*) FROM inventory_clean;

SELECT COUNT(*) FROM inventory_invalid;
```

4.5 Inventory Alert Generation

```
CREATE TABLE inventory_alerts AS SELECT distributor_id,
product_id, closing_stock, reorder_point, CASE WHEN
closing_stock = 0 THEN 'CRITICAL' WHEN closing_stock <
reorder_point THEN 'LOW_STOCK' ELSE 'OK' END AS alert_type
FROM inventory_clean;

SELECT * FROM inventory_alerts WHERE alert_type != 'OK';
```



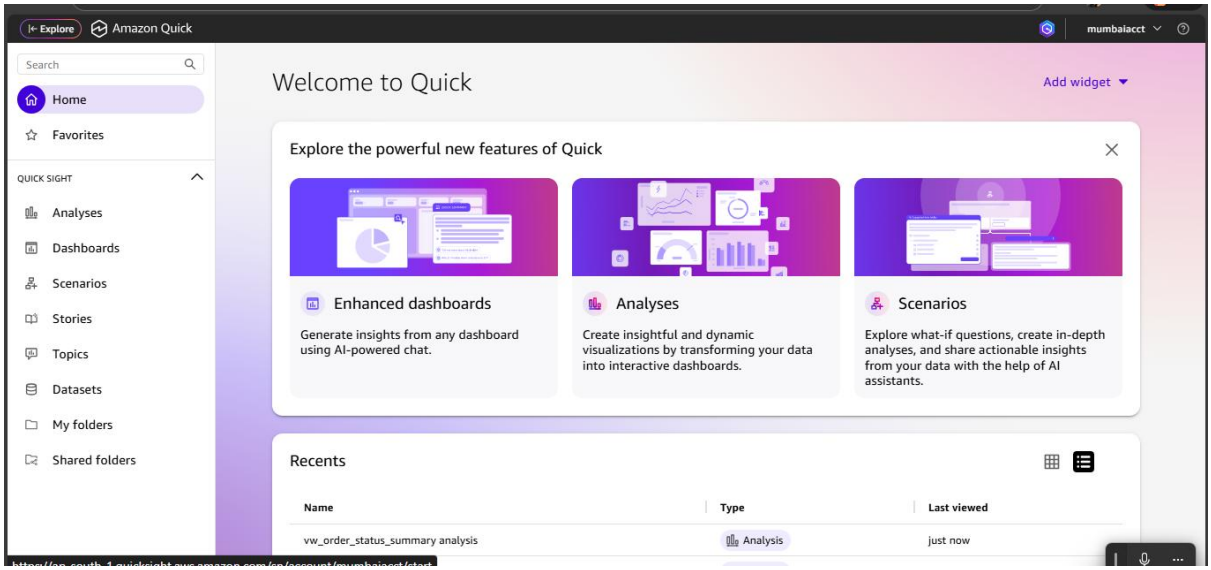

```
[hadoop@ip-172-31-30-184 ~]$ vi process_orders.py
```

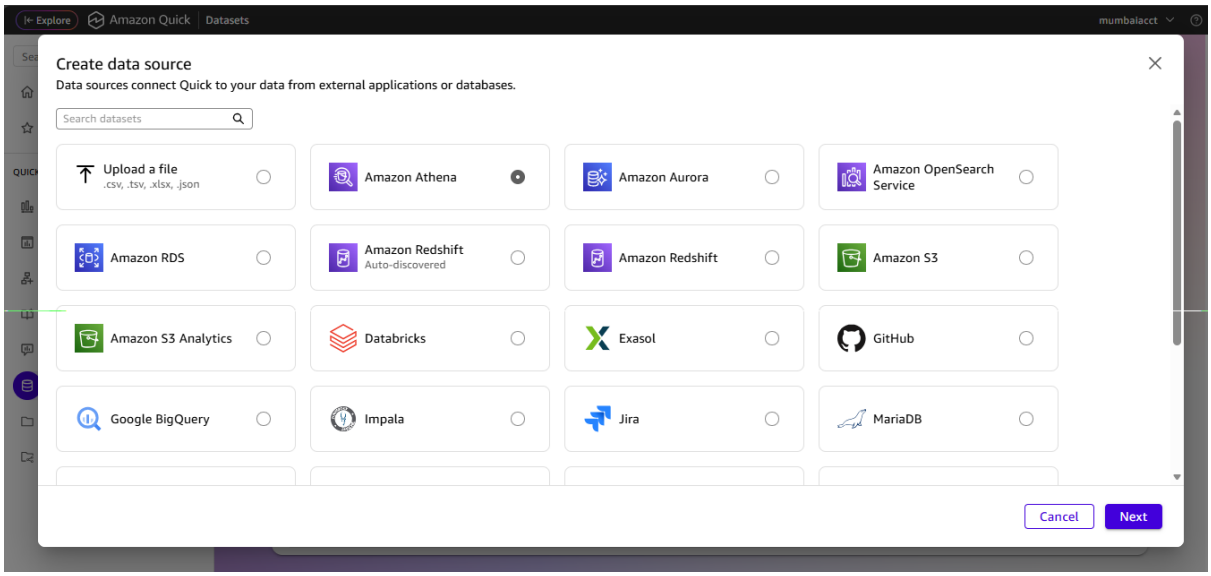
```
from pyspark.sql import SparkSession

from pyspark.sql.functions import col, when, current_date, to_timestamp


def main():


    spark = SparkSession.builder \

        .appName("RetailPulseETL_Kritika") \

        .getOrCreate()


    # ========================

    # 1. READ FROM S3 JSON

    # ========================

    df_json = (

        spark.read

            .option("multiLine", "true")   # JSON is array

            .option("mode", "PERMISSIVE")

            .option("columnNameOfCorruptRecord", "_corrupt_record")
```

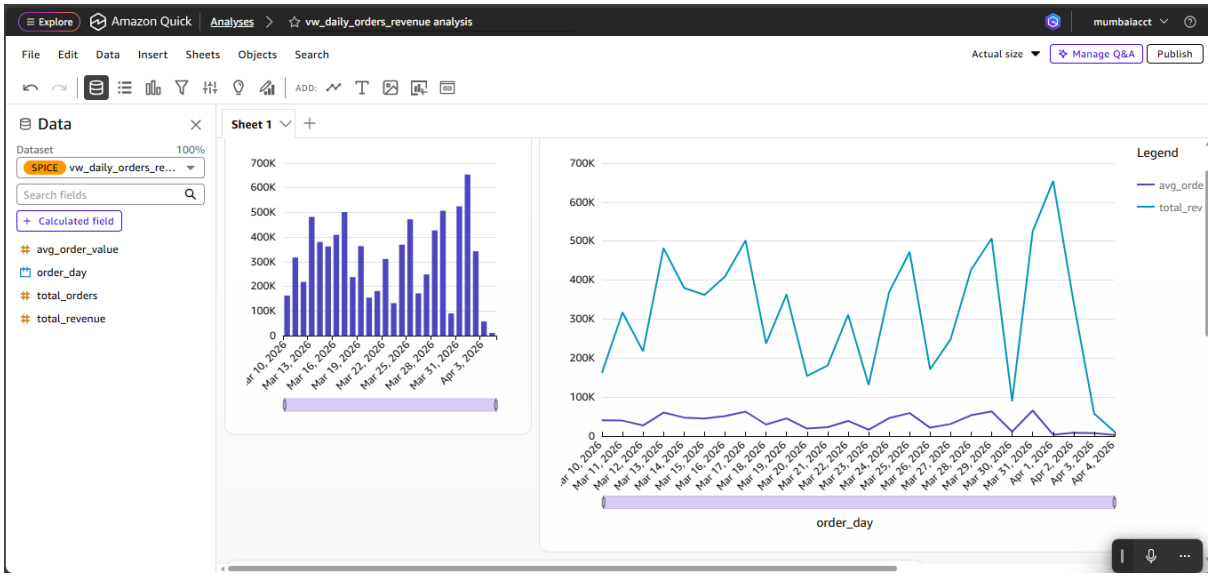
```
    .json("s3://retailpulse-kritika/raw/orders_events.json")
)

print("Schema:")
df_json.printSchema()

# ==========================
# 2. HANDLE CORRUPT RECORDS
# ==========================
if "_corrupt_record" in df_json.columns:
  df_json_good = df_json.filter(col("_corrupt_record").isNull())
  df_json_bad  = df_json.filter(col("_corrupt_record").isNotNull())

  # Store bad records in quarantine
  df_json_bad.write \
    .mode("overwrite") \
    .parquet("s3://retailpulse-kritika/quarantine/raw_orders/")

else:
  df_json_good = df_json

# ==========================
# 3. BASIC CLEANING + STANDARDIZATION
# ==========================
df_clean = df_json_good.select(
  col("event_id"),
  col("order_id"),
  col("event_type"),
```

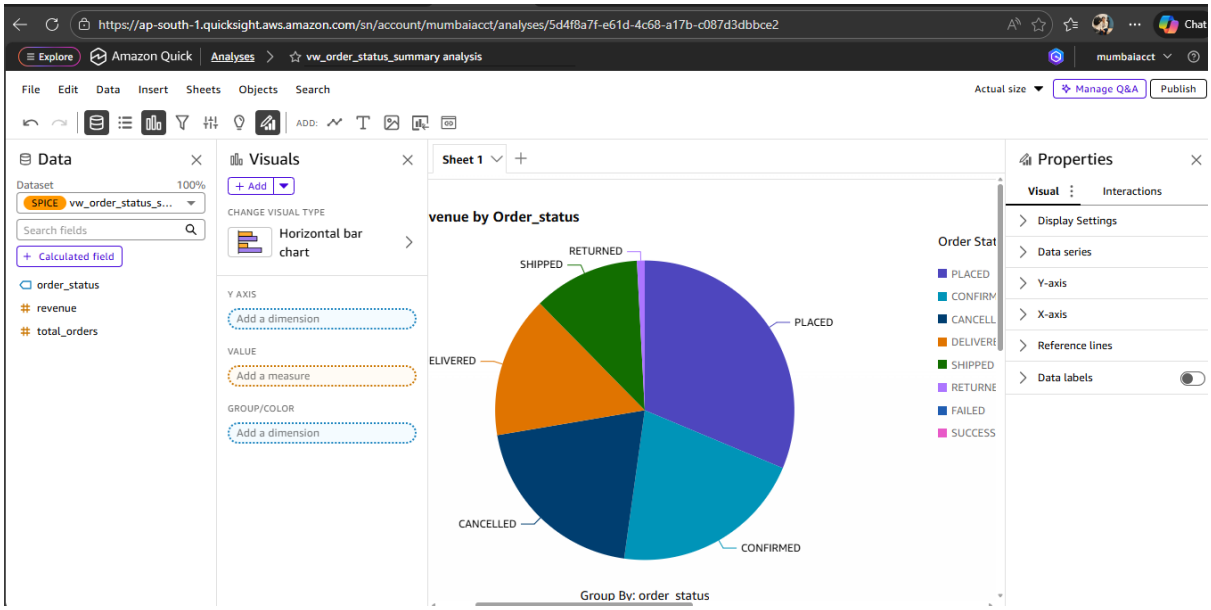
```
    to_timestamp(col("event_timestamp")).alias("event_time"),
    col("customer_id"),
    col("product_id"),
    col("quantity").cast("int"),
    col("status")
)

# ========================
# 4. VALIDATIONS
# ========================
valid_condition = (
    (col("order_id").isNotNull()) &
    (col("event_time") <= current_date())
)

df_valid = df_clean.filter(valid_condition)
df_quarantine = df_clean.filter(~valid_condition)

# ========================
# 5. ENRICHMENT
# ========================
df_enriched = df_valid.withColumn(
    "order_size",
    when(col("quantity") >= 5, "BULK")
    .when(col("quantity") >= 2, "MEDIUM")
    .otherwise("SMALL")
)
```

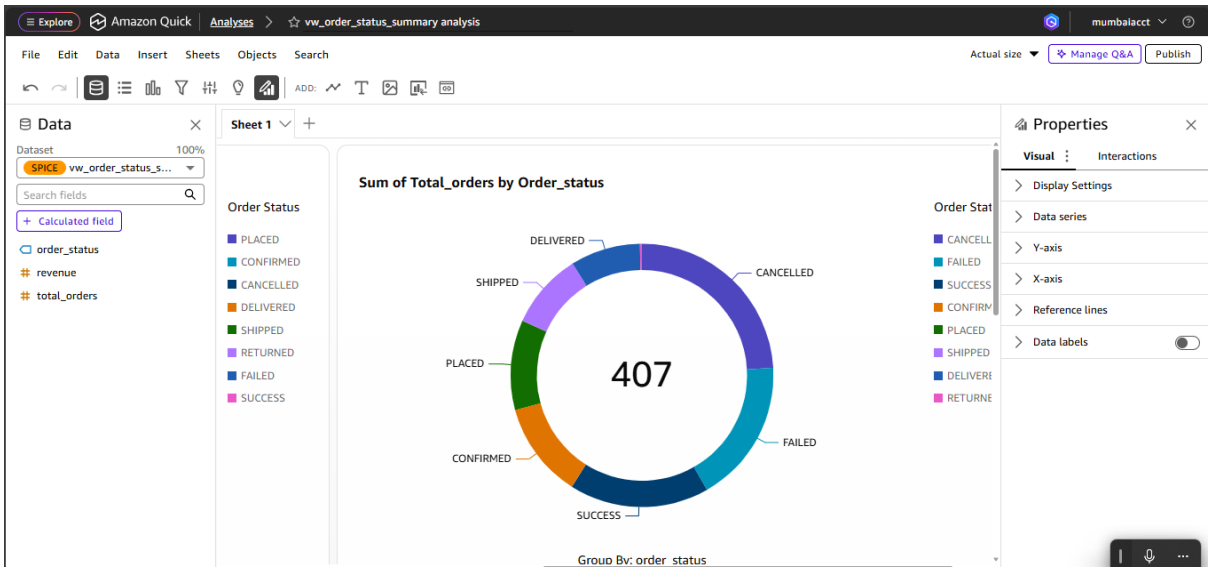
```
    # ========================
    # 6. FILTER SUCCESS EVENTS
    # ========================
    df_final = df_enriched.filter(col("status") == "SUCCESS")

    # ========================
    # 7. WRITE PROCESSED DATA
    # ========================
    df_final.write \
        .mode("overwrite") \
        .parquet("s3://retailpulse-kritika/processed/orders/")

    # ========================
    # 8. WRITE FAILED VALIDATION
    # ========================
    df_quarantine.write \
        .mode("overwrite") \
        .parquet("s3://retailpulse-kritika/quarantine/invalid_orders/")

    spark.stop()


if __name__ == "__main__":
    main()
```


Explore
Amazon Quick
mumbaiacct
Search
Home
Favorites
QUICK SIGHT
Analyses
Dashboards
Scenarios
Stories
Topics
Datasets
My folders
Shared folders
Welcome to Quick
Add widget
Explore the powerful new features of Quick
Enhanced dashboards
Generate insights from any dashboard using AI-powered chat.
Analyses
Create insightful and dynamic visualizations by transforming your data into interactive dashboards.
Scenarios
Explore what-if questions, create in-depth analyses, and share actionable insights from your data with the help of AI assistants.
Recents
Name
Type
Last viewed
vw_order_status_summary analysis
Analysis
just now


Explore
Amazon Quick
Datasets
mumbaiacct
Create data source
Data sources connect Quick to your data from external applications or databases.
Search datasets
Upload a file
.csv, .tsv, .xlsx, .json
Amazon Athena
Amazon Aurora
Amazon OpenSearch Service
Amazon RDS
Amazon Redshift
Auto-discovered
Amazon Redshift
Amazon S3
Amazon S3 Analytics
Databricks
Exasol
GitHub
Google BigQuery
Impala
Jira
MariaDB
Cancel
Next


Explore
Amazon Quick
Analyses
vw_daily_orders_revenue analysis
mumbaiacct
File
Edit
Data
Insert
Sheets
Objects
Search
Actual size
Manage Q&A
Publish
Data
Dataset
100%
SPICE
vw_daily_orders_re...
Search fields
Calculated field
avg_order_value
order_day
total_orders
total_revenue
Sheet 1
700K
600K
500K
400K
300K
200K
100K
0
Legend
avg_orde
total_rev
order_day


https://ap-south-1.quicksight.aws.amazon.com/sn/account/mumbaiacct/analyses/5d4f8a7f-e61d-4c68-a17b-c087d3dbbce2
Explore
Amazon Quick
Analyses
vw_order_status_summary analysis
mumbaiacct
File
Edit
Data
Insert
Sheets
Objects
Search
Actual size
Manage Q&A
Publish
Data
Dataset
100%
SPICE vw_order_status_s...
Search fields
Calculated field
order_status
revenue
total_orders
Visuals
Add
CHANGE VISUAL TYPE
Horizontal bar chart
Y AXIS
Add a dimension
VALUE
Add a measure
GROUP/COLOR
Add a dimension
Sheet 1
venue by Order_status
RETURNED
SHIPPED
PLACED
ELIVERED
CANCELLED
CONFIRMED
Group By: order_status
Order Stat
PLACED
CONFIRM
CANCELL
DELIVERE
SHIPPED
RETURNE
FAILED
SUCCESS
Properties
Visual
Interactions
Display Settings
Data series
Y-axis
X-axis
Reference lines
Data labels


Explore
Amazon Quick
Analyses
vw_order_status_summary analysis
mumbaiacct
File
Edit
Data
Insert
Sheets
Objects
Search
Actual size
Manage Q&A
Publish
Data
Dataset
100%
SPICE vw_order_status_s...
Search fields
Calculated field
order_status
revenue
total_orders
Sheet 1
Order Status
PLACED
CONFIRMED
CANCELLED
DELIVERED
SHIPPED
RETURNED
FAILED
SUCCESS
Sum of Total_orders by Order_status
DELIVERED
SHIPPED
PLACED
CONFIRMED
SUCCESS
FAILED
CANCELLED
407
Group By: order_status
Order Stat
CANCELL
FAILED
SUCCESS
CONFIRM
PLACED
SHIPPED
DELIVERE
RETURNE
Properties
Visual
Interactions
Display Settings
Data series
Y-axis
X-axis
Reference lines
Data labels

Amazon Athena > Query editor

Query in Amazon SageMaker Unified Studio

Use your IAM role to analyze and build with your existing AWS resources in a single data and AI development environment.

Set up

Editor | Recent queries | Saved queries | Settings

Workgroup: primary

Data

Data source: AwsDataCatalog

Catalog: None

Database: retail-kritika-db

Tables and views — Create

Filter tables and views

Query 1 | Query 2 | Query 3 | Query 4 | Query 5

```
CREATE OR REPLACE VIEW vw_top_products AS
SELECT
    p.product_name,
    SUM(o.quantity * o.unit_price_inr) AS revenue,
    COUNT(DISTINCT o.order_id) AS total_orders
FROM final_orders o
JOIN products_clean p
    ON o.product_id = p.product_id
GROUP BY p.product_name;
```

Views (6) 1

- vw_customer_segment_revenue
- vw_daily_orders_revenue
- vw_order_status_summary
- vw_payment_contribution
- vw_payment_revenue
- vw_top_products

Amazon S3 > Buckets > retailpulse-kritika

Successfully created folder "athena-results".

Objects (1/7)

Copy S3 URI | Copy URL | Download | Open | Delete | Actions | Create folder | Upload

Objects are the fundamental entities stored in Amazon S3. You can use Amazon S3 inventory to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. Learn more

Find objects by prefix

Name	Type	Last modified	Size	Storage class
alerts/	Folder	-	-	-
athena-results/	Folder	-	-	-
clean/	Folder	-	-	-
invalid/	Folder	-	-	-
processed/	Folder	-	-	-
quarantine/	Folder	-	-	-
raw/	Folder	-	-	-

aws | Search [Alt+S] | Asia Pacific (Mumbai) | Account ID: 7761-3744-7724 | user10

Lambda > Functions > athena-alert-processor-kritika

athena-alert-processor-kritika

Throttle | Copy ARN | Actions

Function overview Info

Export to Infrastructure Composer | Download

Diagram | Template

athena-alert-processor-kritika

Layers (0)

+ Add trigger

+ Add destination

Function ARN
arn:aws:lambda:ap-south-1:776137447724:function:athena-alert-processor-kritika

Description
-

Last modified
5 minutes ago

Code | Test | Monitor | Configuration | Aliases | Versions

Code source Info

Open in Visual Studio Code | Upload from

athena-alert-processor-kritika

aws | Search [Alt+S] | Asia Pacific (Mumbai) | Account ID: 7761-3744-7724 | user10

Amazon SQS > Queues > athena-alert-queue

athena-alert-queue

Edit | Delete | Purge | Send and receive messages | Start DLQ redrive

Details Info

Name
athena-alert-queue

Type
Standard

ARN
arn:aws:sqs:ap-south-1:776137447724:athena-alert-queue

Encryption
Amazon SQS key (SSE-SQS)

URL
https://sqs.ap-south-1.amazonaws.com/776137447724/athena-alert-queue

Dead-letter queue
-

More

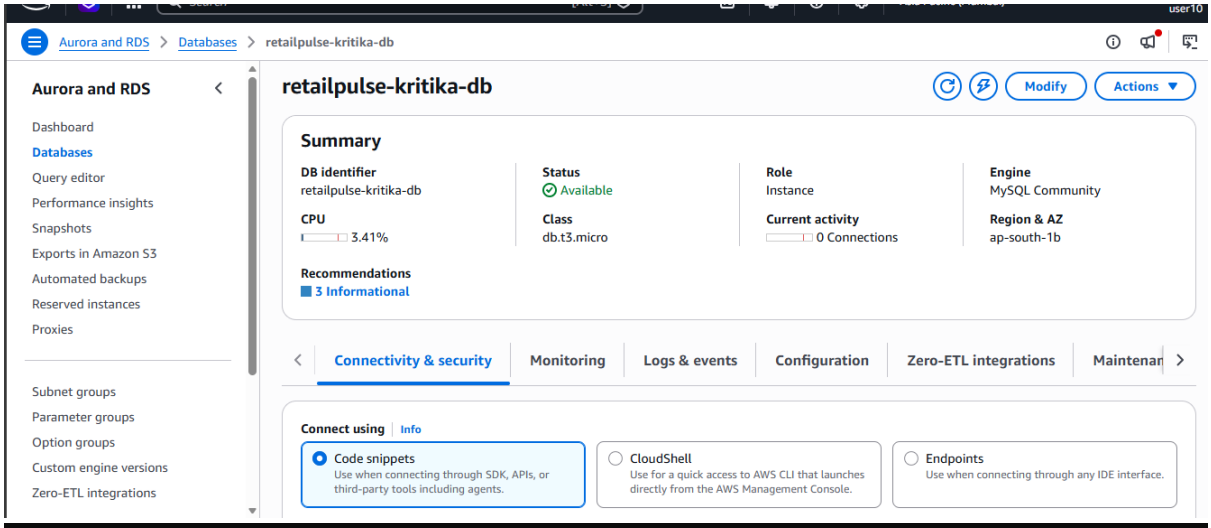
Queue policies | Monitoring | SNS subscriptions | Lambda triggers | EventBridge Pipes | Dead-letter queue | Tagging | Encryption

Subscription region
ap-south-1

PROBLEMS | OUTPUT | CODE REFERENCE LOG | TERMINAL

Execution Results

```
Status: Succeeded
Test Event Name: e

Response:
{
  "status": "Alert Scanned"
}

The area below shows the last 4 KB of the execution log.

Function Logs:
START RequestId: 190d7606-0021-4972-ac22-a66202fccbc2 Version: $LATEST
END RequestId: 190d7606-0021-4972-ac22-a66202fccbc2
REPORT RequestId: 190d7606-0021-4972-ac22-a66202fccbc2  Duration: 7384.71 ms    Billed Duration: 7731 ms    Memory Size: 128 MB Max Memory Used: 91 MB
Init Duration: 345.86 ms
```




```
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MySQL [(none)]> show databases;
+--------------------+
| Database           |
+--------------------+
| information_schema |
| mysql              |
| performance_schema |
| retailpulse_kritika |
| sys                |
+--------------------+
5 rows in set (0.009 sec)

MySQL [(none)]> use retailpulse_kritika;
```

```
Database changed
MySQL [retailpulse_kritika]> show tables;
+-------------------------------+
| Tables_in_retailpulse_kritika |
+-------------------------------+
| customers                     |
| customers_clean               |
| customers_invalid             |
| distributors                  |
| inventory_alerts              |
| inventory_clean               |
| inventory_invalid             |
| inventory_snapshots           |
| orders                        |
| orders_clean                  |
| orders_invalid                |
| products                      |
| products_clean                |
| products_invalid              |
+-------------------------------+
14 rows in set (0.002 sec)
```

```
MySQL [retailpulse_kritika]> describe customers;
+---------------------+-------------+------+-----+---------+-------+
| Field               | Type        | Null | Key | Default | Extra |
+---------------------+-------------+------+-----+---------+-------+
| customer_id         | varchar(30) | YES  |     | NULL    |       |
| registration_date   | date        | YES  |     | NULL    |       |
| city                | varchar(80) | YES  |     | NULL    |       |
| state               | varchar(50) | YES  |     | NULL    |       |
| pincode             | varchar(10) | YES  |     | NULL    |       |
| customer_segment    | varchar(20) | YES  |     | NULL    |       |
| email_verified      | tinyint(1)  | YES  |     | NULL    |       |
| phone_verified      | tinyint(1)  | YES  |     | NULL    |       |
| loyalty_tier        | varchar(15) | YES  |     | NULL    |       |
| acquisition_channel | varchar(30) | YES  |     | NULL    |       |
| date_of_birth       | date        | YES  |     | NULL    |       |
| gender              | varchar(10) | YES  |     | NULL    |       |
+---------------------+-------------+------+-----+---------+-------+
12 rows in set (0.013 sec)

MySQL [retailpulse_kritika]> describe distributors;
+------------------+--------------+------+-----+---------+-------+
| Field            | Type         | Null | Key | Default | Extra |
+------------------+--------------+------+-----+---------+-------+
| distributor_id   | varchar(20)  | YES  |     | NULL    |       |
| distributor_name | varchar(150) | YES  |     | NULL    |       |
| region           | varchar(30)  | YES  |     | NULL    |       |
| state            | varchar(50)  | YES  |     | NULL    |       |
| city             | varchar(80)  | YES  |     | NULL    |       |
| tier             | varchar(15)  | YES  |     | NULL    |       |
| contact_email    | varchar(100) | YES  |     | NULL    |       |
| contact_phone    | varchar(20)  | YES  |     | NULL    |       |
| is_active        | tinyint(1)   | YES  |     | NULL    |       |
| onboarding_date  | date         | YES  |     | NULL    |       |
+------------------+--------------+------+-----+---------+-------+
```

```
MySQL [retailpulse_kritika]> describe  products;
+--------------------+---------------+------+-----+---------+-------+
| Field              | Type          | Null | Key | Default | Extra |
+--------------------+---------------+------+-----+---------+-------+
| product_id         | varchar(30)   | YES  |     | NULL    |       |
| product_name       | varchar(200)  | YES  |     | NULL    |       |
| category           | varchar(50)   | YES  |     | NULL    |       |
| sub_category       | varchar(80)   | YES  |     | NULL    |       |
| brand              | varchar(80)   | YES  |     | NULL    |       |
| base_mrp_inr       | decimal(10,2) | YES  |     | NULL    |       |
| cost_price_inr     | decimal(10,2) | YES  |     | NULL    |       |
| weight_kg          | decimal(6,3)  | YES  |     | NULL    |       |
| is_active          | tinyint(1)    | YES  |     | NULL    |       |
| launch_date        | date          | YES  |     | NULL    |       |
| reorder_threshold  | int           | YES  |     | NULL    |       |
| return_window_days | int           | YES  |     | NULL    |       |
+--------------------+---------------+------+-----+---------+-------+
12 rows in set (0.002 sec)

MySQL [retailpulse_kritika]> describe inventory_snapshots;
+------------------+-------------+------+-----+---------+-------+
| Field            | Type        | Null | Key | Default | Extra |
+------------------+-------------+------+-----+---------+-------+
| snapshot_id      | varchar(40) | YES  |     | NULL    |       |
| snapshot_date    | date        | YES  |     | NULL    |       |
| distributor_id   | varchar(20) | YES  |     | NULL    |       |
| product_id       | varchar(30) | YES  |     | NULL    |       |
| opening_stock    | int         | YES  |     | NULL    |       |
| units_received   | int         | YES  |     | NULL    |       |
| units_dispatched | int         | YES  |     | NULL    |       |
| units_returned   | int         | YES  |     | NULL    |       |
| units_damaged    | int         | YES  |     | NULL    |       |
| closing_stock    | int         | YES  |     | NULL    |       |
| reorder_point    | int         | YES  |     | NULL    |       |
+------------------+-------------+------+-----+---------+-------+
11 rows in set (0.002 sec)
```

```
The authenticity of host 'ec2-43-205-134-185.ap-south-1.compute.amazonaws.com (43.205.134.185)' can't be established.
ED25519 key fingerprint is SHA256:gKkT0YnDSknSYW1Xs9G+VAhyHvroMQQpNYTrhmrVg1s.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'ec2-43-205-134-185.ap-south-1.compute.amazonaws.com' (ED25519) to the list of known hosts.
   ,     #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \_#####\
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
Last login: Tue Apr  7 05:22:08 2026

EEEEEEEEEEEEEEEEEEEE MMMMMMMM           MMMMMMMM RRRRRRRRRRRRRRR
E::::::::::::::::::E M::::::M         M::::::M R::::::::::::::R
EE:::::EEEEEEEEE:::E M::::::::M       M::::::::M R:::::RRRRRR:::::R
  E::::E       EEEEE M:::::::::M     M:::::::::M RR::::R      R::::R
  E::::E             M:::::M::M   M::M:::::M   R::R      R::::R
  E::::EEEEEEEEEE    M:::::M M::M M::M M:::::M   R::RRRRRR:::::R
  E::::::::::::::E   M:::::M  M::M::M  M:::::M   R::::::::::RR
  E::::EEEEEEEEEE    M:::::M   M:::M   M:::::M   R::RRRRRR:::R
  E::::E             M:::::M    M:::M    M:::::M   R::R      R::::R
  E::::E       EEEEE M:::::M     MMM     M:::::M   R::R      R::::R
EE:::::EEEEEEEE:::E M:::::M             M:::::M   R::R      R::::R
E::::::::::::::::::E M:::::M             M:::::M RR::::R      R::::R
EEEEEEEEEEEEEEEEEEEE MMMMMMM             MMMMMMM RRRRRRR      RRRRRR

[hadoop@ip-172-31-30-184 ~]$ aws s3 ls s3://retailpulse-kritika/raw/
2026-04-06 12:00:55          0
2026-04-06 13:06:49      48981 orders_events.json
[hadoop@ip-172-31-30-184 ~]$ nano process_orders.py
```

```
C:\Users\kritika.nimje\Downloads>vi process_orders.py
```

```
PS C:\Users\kritika.nimje\Downloads> scp -i "Kritika_ec2.pem" mysql-connector-java-8.0.22.jar hadoop@13.204.70.132:/home/hadoop/
mysql-connector-java-8.0.22.jar                                                    100% 2333KB   5.1MB/s   00:00
```